实验十一 I^2C EEPROM 读写系统的设计

要求：利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个 I^2C EEPROM 读写的系统，按键选择实验板键盘阵列中的任意一个数字，将其写入实验板上的 EEPROM 24LC02 中，并显示在数码管上；掉电后重新上电，再从实验板上的 EEPROM 24LC02 中读取写入的数字，将其显示在数码管上，并可适当添加发挥功能。

重点：I^2C 总线双向传输协议的格式及其应用，以及数码管扫描显示、键盘扫描及按键消抖技术。

在学习这个实验时希望大家能多多在网上查找相关资料，看看别人的实现方法，比较一下各种实现的思路，这在学习和开发当中都是非常必要的。开发人员不是理论家，绝不能闭门造车，作为一个开发人员，你的使命是以最快的速度实现你的设计，能借鉴别人的地方一定要去用，要学会站在巨人的肩膀上；但同时别人的东西也绝对不是完美的，你必须读懂它，然后再去使用它、改进它。

- **功能现象：**

按动实验板键盘阵列中的任意一个数字键，随后再按动实验板上的独立按键 A，这时将刚才选择的数字写入实验板上的 EEPROM 24LC02 中，同时将该数字显示在数码管上。为了验证数据是否真正地写入了 EEPROM 里，这时关闭实验板电源，等待一会儿再开启电源，按动实验板上的独立按键 B，这时从实验板上的 EEPROM 24LC02 中读取数据，同时将该所读数据（应该即为刚才键入的数字）显示在数码管上，以表明数据的确存入了 EEPROM，而且确实是掉电保持的。也就是说独立按键 A 就是写命令，独立按键 B 就是读命令。

注意：首先请确认实验板上的拨码开关均拨向数字“1”、“2”、“3”、“4”一端，因为 EEPROM 与拨码开关共享了相同的 CPLD 引脚。只要将拨码开关拨向数字一端也就不会影响 EEPROM 了。

- **实现原理：**

1、I^2C 总线的基本概念

I^2C 总线（Inter-Integrated Circuit Bus，中文翻译成内置集成电路总线或内部整合电路总线），是 20 世纪 80 年代由 PHILIPS 公司为了让主板、嵌入式系统或手机能连接低速周边装置而开发的一种两线式串行通讯总线，使用多主从架构，主要用于连接微控制器及其外围设备，是微电子通信控制领域广泛采用的一种总线标准。它是同步通信的一种特殊形式，具有接口线少、控制方式简单、器件封装形式小、通信速率较高等优点。I^2C 的正确读法为“I-squared-C”，而“I-two-C”则是另一种错误但被广泛使用的读法，在中国则多以“I 方 C”称之。其基本概念包括：

- ✓ 发送器（Transmitter）：发送数据到总线的器件
- ✓ 接收器（Receiver）：从总线接收数据的器件
- ✓ 主机（Master）：初始化发送、产生时钟信号和终止发送的器件（无论读写均为主机终止）
- ✓ 从机（Slave）：被主机寻址的器件

I^2C 总线是双向传输的总线，因此主机和从机都可能成为发送器和接收器。如果主机向从机发送数据，则主机是发送器，而从机是接收器；如果主机从从机读取数据，则主机是接收器，而从机是发送器。不论主机是发送器还是接收器，时钟信号 SCL 都要由主机来产生。

图 9-10 是 I^2C 总线信号连接的示意图。其中，Master 为主机，Slave1 和 Slave2 为从机。SDA 和 SCL 是 I^2C 总线的两根信号线，一根是串行数据线 SDA，另一根是串行时钟线 SCL，通常这两个管脚都是漏极开路（或集电极开路）输出结构，因此在实际使用时，SDA 和 SCL 信号线都必须要加上拉电阻 Rp（Pull-Up Resistor）接到电源 Vdd（如图 9-10 中所示），上拉电阻一般取值 3~10kΩ。开漏结构的好处是：(1)、当总线空闲时，这两条信号线都保持高电平，不会消耗电流；(2)、电气兼容性好。上拉电阻接 5V 电源就能与 5V 逻辑器件进行接口相连，上拉电阻接 3.3V 电源又能与 3.3V 逻辑器件进行接口相连。因为是开漏结构，所以不同器件的 SDA 与 SDA 之间、SCL 与 SCL 之间可以直接相连，不需要额外的转换电路。

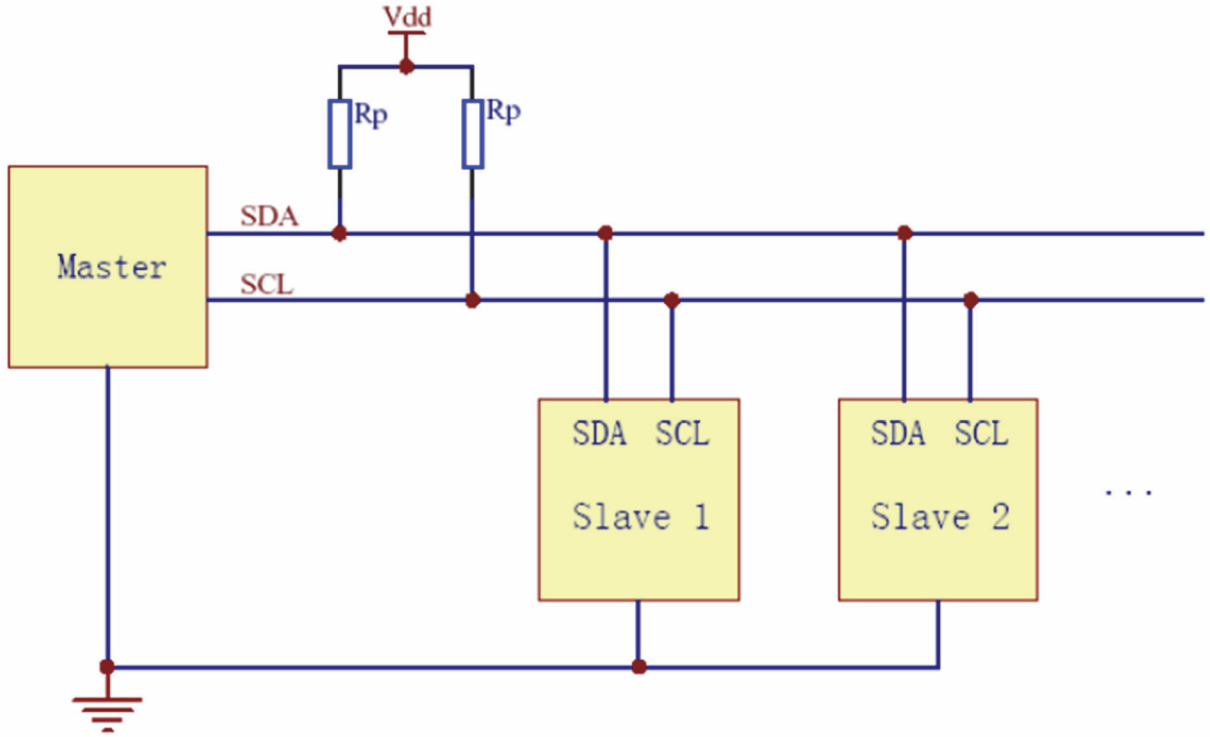


图 9-10 I²C 总线信号连接示意图

2、EEPROM 器件：24LC02

EEPROM 器件通常都是兼容的，24LC02、24LC04 一直到 24LC256 管脚都是兼容的，其管脚图如图 9-11 所示。其中，A0、A1、A2 是从机地址，因为总线上如果有多个 EEPROM 器件，这时就要通过地址引脚加以区分；Vcc 是正电源，根据型号不同，电压范围也不同，比如 24LC02 电压范围为 2.5V~5.5V，24AA02 电压范围为 1.8V~5.5V；WP 为写保护使能引脚（Write Protect），当其为高电平时，主机就只能读而不能写了（即处于写保护状态）。

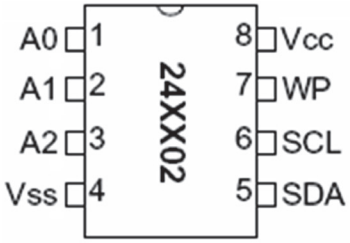


图 9-11 EEPROM 器件的管脚图

3、I²C 总线的通信速率

I²C 总线的通信速率受主机控制，能快能慢。但是最高速率是有限制的，I²C 总线上数据的传输速率在标准模式下为 100kbps，在快速模式下为 400kbps。按照后来修订的版本，位速率最高可达 3.4Mbps。

4、I²C 总线上数据的有效性（Data validity）

I²C 总线上数据有效性的示意图如图 9-12 所示。数据线 SDA 的电平状态必须在时钟线 SCL 处于高电平期间保持稳定不变。SDA 的电平状态只有在 SCL 处于低电平期间才允许改变，但在 I²C 总线的起始和结束时例外。注：某些其它的串行总线协议，如 SPI（Serial Peripheral Interface，串行外设接口），可能规定数据在时钟信号的边沿（上升沿或下降沿）有效，而 I²C 总线则是电平有效。

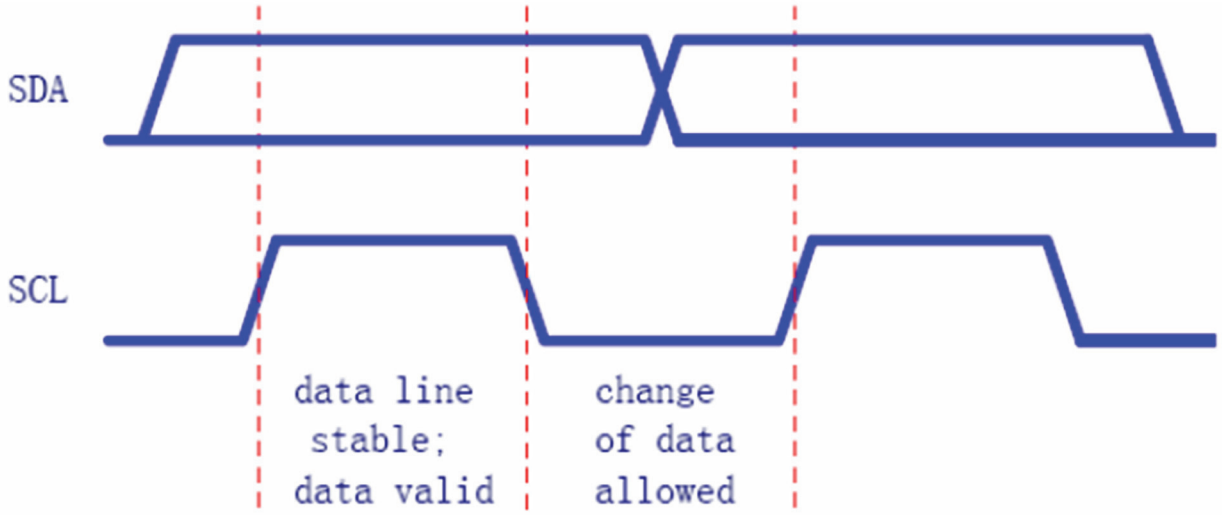


图 9-12 I²C 总线上数据有效性的示意图

5、起始条件和停止条件（START and STOP conditions）

I²C 总线数据传输的起始条件和停止条件示意图如图 9-13 所示。

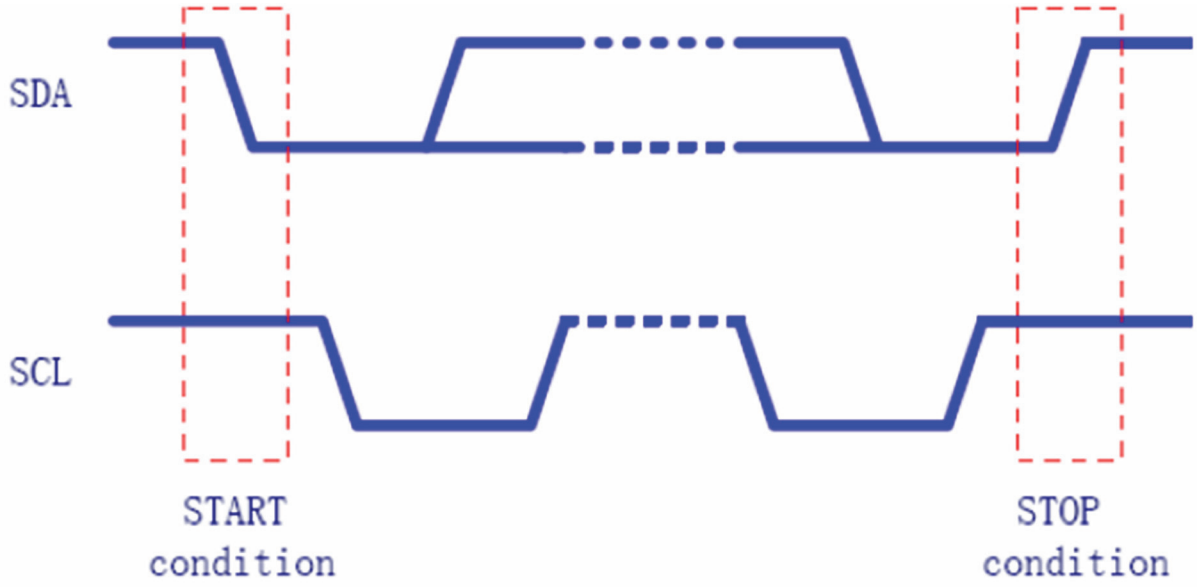


图 9-13 I²C 起始条件和停止条件示意图

- ✓ 起始条件：当 SCL 处于高电平期间，SDA 从高电平向低电平跳变时产生起始条件。总线在起始条件产生后便处于忙的状态。起始条件常简记为 S（即 START）。
- ✓ 停止条件：当 SCL 处于高电平期间时，SDA 从低电平向高电平跳变时产生停止条件。总线在停止条件产生后处于空闲状态。停止条件常简记为 P（即 STOP）。

6、从机地址（Slave Address）

I²C 总线不需要额外的地址译码器和片选信号。多个具有 I²C 总线接口的器件都可连接到同一条 I²C 总线上，它们之间通过器件地址来区分。主机是主控器件，它不需要器件地址；其它器件都属于从机，需有器件地址。必须保证同一条 I²C 总线上所有从机的地址都是唯一确定的，不能有重复，否则 I²C 总线将不能正常工作。一般从机地址由 7 位地址位和 1 位读写标志 R/W 组成，7 位地址占据高 7 位，读写位在最后，如图 9-14 所示。读写位是 0，表示主机将要向从机写入数据；读写位是 1，则表示主机将要从从机读取数据。例如一种 EEPROM 器件的控制字节（CONTROL BYTE）分配格式如图 9-14 所示，其中，B1~B0 就是从机地址管脚上的地址信号，1010 是器件类型的分类编号。

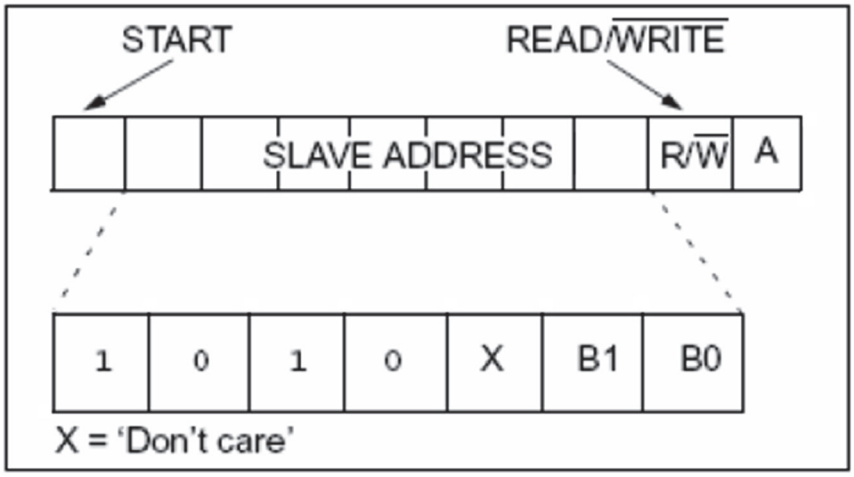


图 9-14 EEPROM 器件的控制字节（CONTROL BYTE）分配格式示意图

7、数据是按字节传输的

I^2C 总线总是以字节(Byte)为单位收发数据。每次传输的字节数量没有严格限制。首先传输的是数据的最高位MSB，第 7 位)，最后传输的是最低位（LSB，第 0 位)。另外，每个字节之后还要跟一个响应位，称为应答。

8、应答（Acknowledge）

在 I^2C 总线传输数据过程中，每传输一个字节，都要在其后跟上一个应答状态位。接收器接收数据的情况可以通过应答位来告知发送器。应答位的时钟脉冲仍由主机产生，而应答位的数据状态则遵循“谁接收谁产生”的原则，即总是由接收器产生应答位，也就是说，主机向从机发送数据时，应答位由从机产生；主机从从机接收数据时，应答位由主机产生。I^2C 总线标准规定：应答位为 0，表示接收器应答（ACK），常简记为 A；应答位为 1，则表示非应答（NACK），常简记为 $\overline{A}$。发送器发送完 LSB 之后，应当释放 SDA 线（拉高 SDA，输出晶体管截止），以等待接收器产生应答位。如果在接收器接收完最后一个字节的数据，或者不能再接收更多的数据时，应当产生非应答来通知发送器。发送器如果发现接收器产生了非应答状态，则应当终止发送。

9、基本的数据传输格式

基本的数据传输格式如图 9-15、图 9-16 所示。



图 9-15 主机向从机发送数据的基本格式



图 9-16 主机从从机接收数据的基本格式

其中，

S：起始位（START）

SA：从机地址（Slave Address），7 位从机地址

W：写标志位（Write），1 位写标志

R：读标志位（Read），1 位读标志

A：应答位（Acknowledge），1 位应答

$\overline{A}$：非应答位（Not Acknowledge），1 位非应答

D：数据（Data），每个数据都必须是 8 位

P：停止位（STOP）

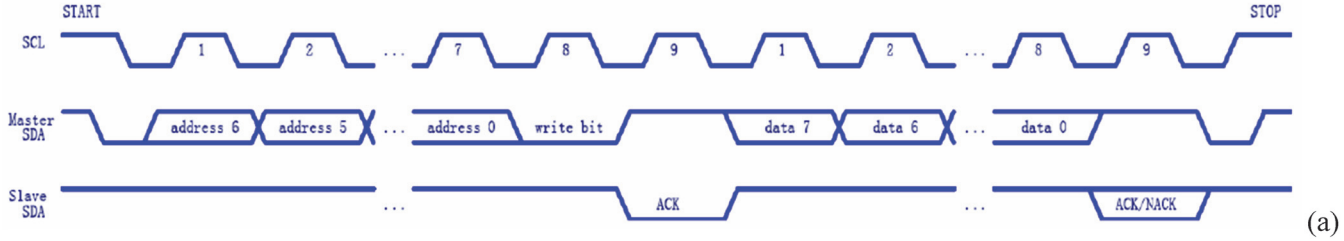
阴影：主机产生的信号

无阴影：从机产生的信号

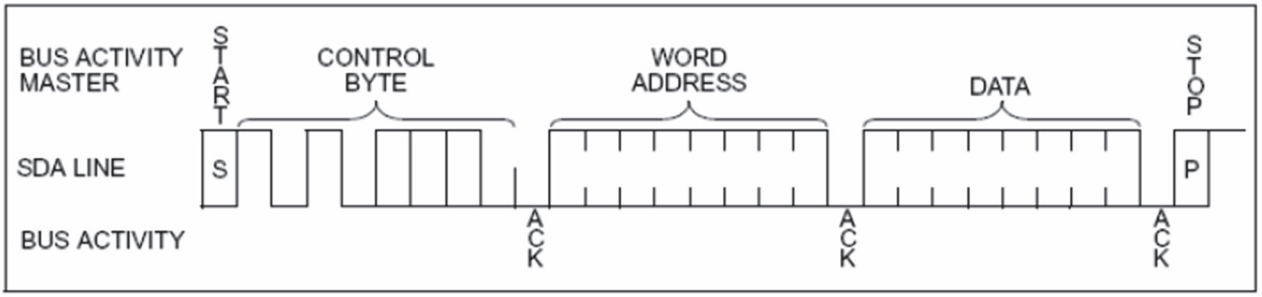
应当注意的是，与图 9-16 中的情况不同的是，在图 9-15 中，主机向从机发送最后一个字节的数据时，从机可能应答也可能非应答，但不管怎样主机都可以产生停止条件。如果主机在向从机发送数据（甚至包括从机地址在内）时检测到从机非应答，则应当及时停止传输。

10、传输一个字节数据的时序图

为了更清楚地了解 I^2C 总线的基本数据传输过程，下面画出了只传输一个字节数据的时序图，如图 9-17、图 9-18 所示（其中的(a)和(b)图实际上是一致的），这是最基本的传输方式。**其中，CONTROL BYTE 即为包含从机地址的控制字节，WORD ADDRESS 即为 EEPROM 中的数据存储地址**。在图 9-17 和图 9-18 中的(a)图里，SDA 信号线被画成了两条，一个是主机产生的（Master SDA），另一个是从机产生的（Slave SDA）。实际上主机和从机的 SDA 信号线总是连接在一起的，是同一根 SDA。在这里画成两个 SDA 主要是有助于深刻理解在 I^2C 总线上主机和从机的不同行为。**另外注意：按照如图 9-18 所示的 I^2C 协议要求，CONTROL BYTE 需要先后写两次。**

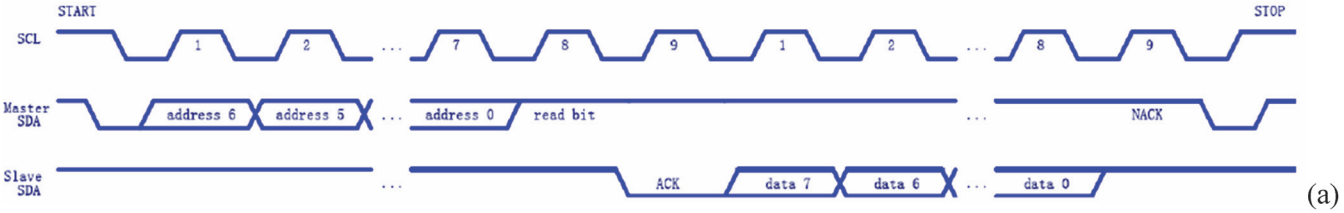


(a)

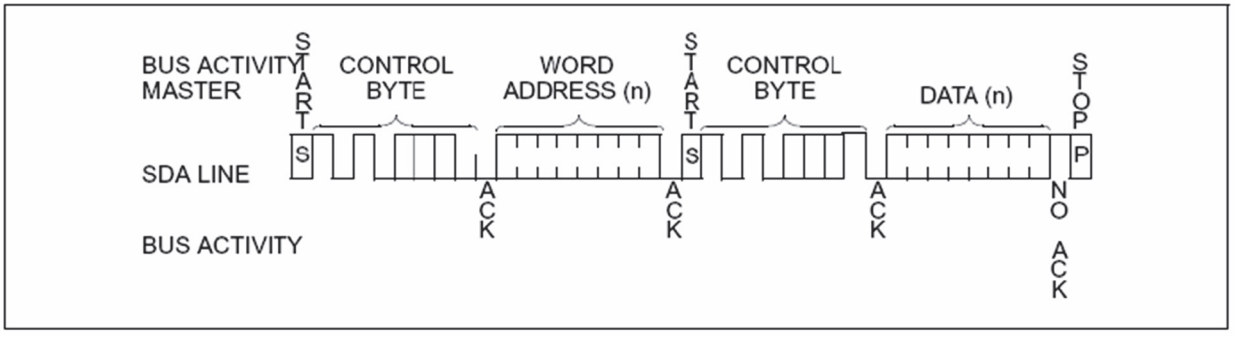


(b)

图 9-17 主机向从机发送 1 个字节数据（即 BYTE WRITE）的时序图



(a)

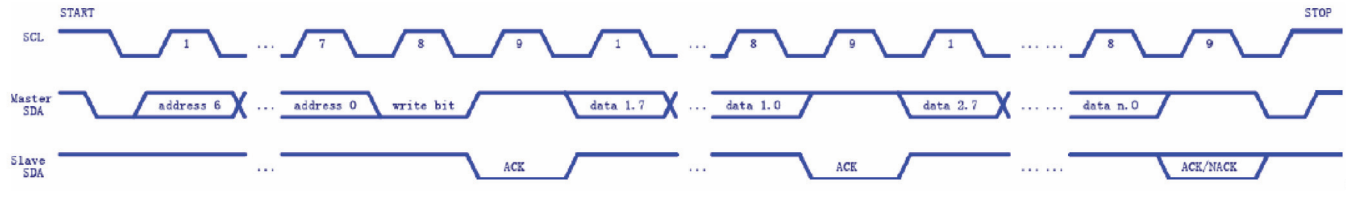


(b)

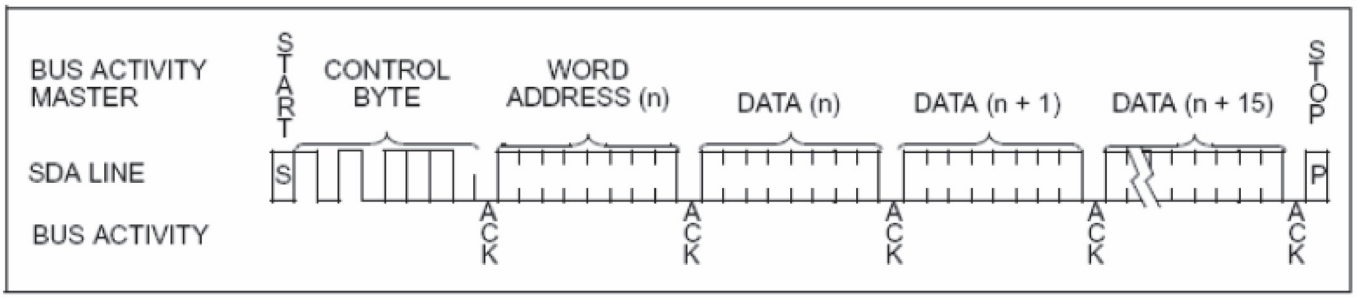
图 9-18 主机从从机接收 1 个字节数据（即 RANDOM READ）的时序图

11、传输多个字节数据的时序图

主机连续向从机发送或从从机接收多个字节数据的情况也很容易理解，其时序图如图 9-19、图 9-20 所示（其中的(a)和(b)图实际上是一致的）。**其中，CONTROL BYTE 即为包含从机地址的控制字节，WORD ADDRESS 即为 EEPROM 中的数据存储地址**。

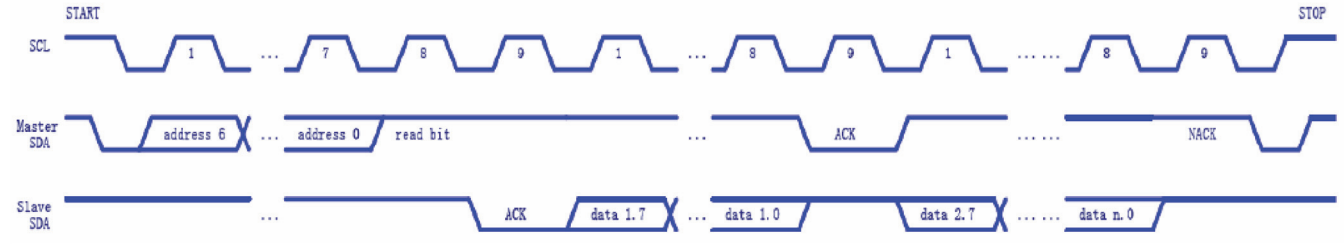


(a)

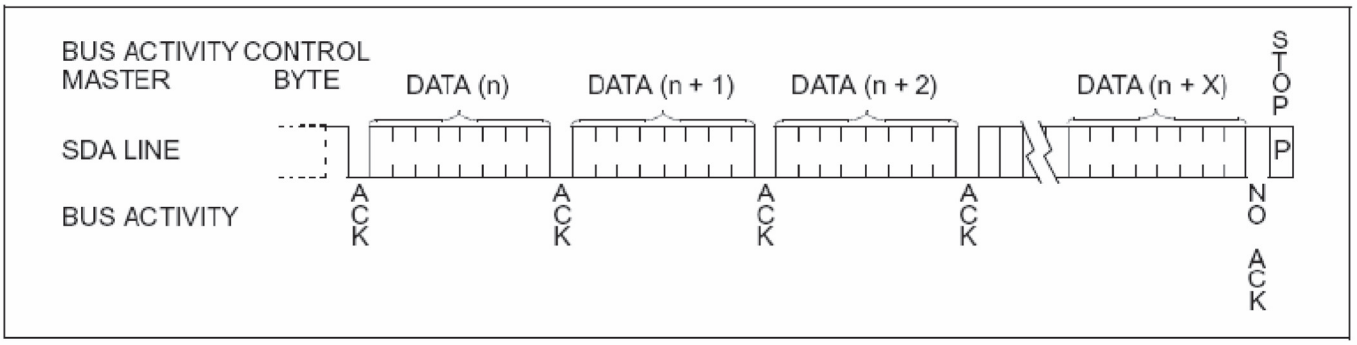


(b)

图 9-19 主机向从机连续发送多个字节数据（即 PAGE WRITE）的时序图



(a)



(b)

图 9-20 主机从从机连续接收多个字节数据（即 SEQUENTIAL READ）的时序

- 部分参考源代码（**Verilog** 语言）：

```
//SDA 为 I2C 数据总线，对应 CPLD 的 5 脚，与实验板上拨码开关 2 复用，拨到数字一侧即不影响 I2C
//SCL 为 I2C 时钟线，对应 CPLD 的 6 脚，与实验板上拨码开关 1 复用，拨到数字一侧即不影响 I2C

//CMD_RD 和 CMD_WR 为按键命令
//CMR_WR 为写，对应独立按键 A；CMR_RD 为读，对应独立按键 B
```

```
//phase0, phase1, phase2, phase3 为四个阶段的状态变量
//phase0 对应 SCL 的上升沿时刻，phase2 对应 SCL 的下降沿时刻
//phase1 对应从 SCL 高电平的中间时刻，phase3 对应从 SCL 低电平的中间时刻
//phase1 为数据的有效时段，此时数据线不允许变化
//phase3 为数据的无效时段，允许更改数据

//参数设置，相当于 C 语言中的宏定义
parameter i2c_addr=7'b1010000,eeprom_addr=8'b00000110;
//i2c_addr 为 EEPROM 器件从机地址
//eeprom_addr 为 EEPROM 存储空间地址，在这里固定存储地址为 00000110

assign SCL=scl_buf; //scl_buf 为 I2C 时钟线缓冲
assign SDA=(Flag_RW)?sda_buf:1'bz; //sda_buf 为 I2C 数据线缓冲
//Flag_RW 为 1 则将 SDA 设置为写状态，为 0 则将 SDA 设置为读状态

//产生采样的 4 个阶段
always @(posedge CLK or negedge RST) //CLK 为 6MHz 时钟源信号
begin
    if(RST==0)
    begin
        phase0 <= 0;
        phase1 <= 0;
        phase2 <= 0;
        phase3 <= 0;
        clk_div<= 0;
    end
    else
    begin
        if(clk_div!=4'b1111) clk_div <= clk_div + 4'b0001;
        else clk_div <= 0;
        //phase0：上升沿
        if(phase0==1) phase0 <= 0;
        else if(clk_div==4'b1111) phase0 <= 1;
        //phase1：‘1’的中间时刻
        if(phase1==1) phase1 <= 0;
        else if(clk_div==4'b0011) phase1 <= 1;
        //phase2：下降沿
        if(phase2==1) phase2 <= 0;
        else if(clk_div==4'b0111) phase2 <= 1;
        //phase3：‘0’的中间时刻
        if(phase3==1) phase3 <= 0;
        else if(clk_div==4'b1011) phase3 <= 1;
    end
end
//总体效果是：4 拍为一个 phase，所以 16 拍经历四个 phase
//0 时是 phase0=1，4 时是 phase1=1，8 时是 phase2=1，12 时是 phase3=1
```

```
//用于按键消抖的延时进程
always @(posedge CLK or negedge RST)
begin
    if(RST==0) key_delay<=0; //key_delay 为按键消抖计数器
    else if(start_delay==1) key_delay<=key_delay+1; // start_delay 为按键消抖计数器计数开始信号，1 有效
    else key_delay<=0;
end

//I2C 进程
always @(posedge CLK or negedge RST)
begin
    if(RST==0)
    begin
        start_delay<=0;
        eeprom_state<=0; //eeprom_state 为操作状态，0 无操作，1 写操作状态，2 读操作状态
    end
    else
        begin
            case(eeprom_state)
                0: //无操作命令状态
                    begin
                        WrData<={4'b0000,DATA_IN};//合并成一个 8 位数
                        //DATA_IN 为输入的 4 位数据；WrData 为待写入 EEPROM 的 8 位数据
                        RdData<=0; //RdData 用于存放从 EEPROM 读出的 8 位数据
                        bit_state<=0; //bit_state 为发送数据或接收数据时的位状态
                        Flag_RW<=0; //读

                        //按键防抖动
                        if((key_delay==0)&&((CMD_WR==0)||(CMD_RD==0)))start_delay<=1;
                        else if(key_delay==10'b1000000000)
                            begin
                                if(CMD_WR==0)eeprom_state<=1; //写操作状态
                                else if(CMD_RD==0)eeprom_state<=2; //读操作状态
                                else eeprom_state<=0; //无操作命令状态
                                start_delay<=0; //停止按键消抖计数器工作
                            end
                    end

                1: //写 EEPROM 操作（其时序参见图 9-17 中的(b)图）
                    begin
                        if(phase0==1)scl_buf<=1;
                        else if(phase2==1)scl_buf<=0;
                        //1 到 0 跳变，产生开始 START 信号
```

```
case(bit_state)
    0: //帧开始，从机地址的第 0 位
      //i2c_addr 从机地址是从高位开始的，这与存储地址和数据发送形式相反
      //具体可参考 24LC02 数据手册
        begin
            if(phase1==1)begin sda_buf<=0;Flag_RW<=1;end
            if((phase3&Flag_RW)==1)
                begin sda_buf<=i2c_addr[6];Flag_RW<=1;bit_state<=1;end
        end
    1: //从机地址的第 1 位
        if(phase3==1)begin sda_buf<=i2c_addr[5];Flag_RW<=1;bit_state<=2;end
    //略
    6: //从机地址的第 6 位
        if(phase3==1)begin sda_buf<=i2c_addr[0];Flag_RW<=1;bit_state<=7;end
    7: //表示写从机（控制字节 CONTROL BYTE 中的 RW’为 0）
        if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=8;end
    8: //准备读应答 ACK
        if(phase3==1)begin Flag_RW<=0;bit_state<=9;end
    9: //读应答 ACK
        begin
            if(phase0==1)sda_tem<=SDA; //读 ACK
            //sda_tem 为 I2C 数据线缓冲，临时存储读 SDA 数据

            if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
            //判断 ACK 信号是否有效，为 1 表示无效，返回初始状态

            if(phase3==1)begin sda_buf<=eeprom_addr[7];Flag_RW<=1;bit_state<=10;end
            //发送数据存储地址 WORD ADDRESS，写地址 bit0
        end
    10: //写地址 bit1
        if(phase3==1)begin sda_buf<=eeprom_addr[6];Flag_RW<=1;bit_state<=11;end
    //略
    16: //写地址 bit7
        if(phase3==1)begin sda_buf<=eeprom_addr[0];Flag_RW<=1;bit_state<=17;end
    17: //准备读 ACK
        if(phase3==1)begin Flag_RW<=0;bit_state<=18;end
    18: //读 ACK，然后开始写数据
        begin
            if(phase0==1)begin sda_tem<=SDA;end
            if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
            //开始写数据[7]
            if(phase3==1)begin sda_buf<=WrData[7];Flag_RW<=1;bit_state<=19;end
        end
    19: //写入数据[6]
        if(phase3==1)begin sda_buf<=WrData[6];Flag_RW<=1;bit_state<=20;end
    //略
```

```
                25: //写入数据[0]
                    if(phase3==1)begin sda_buf<=WrData[0];Flag_RW<=1;bit_state<=26;end
                26: //准备读 ACK
                    if(phase3==1)begin Flag_RW<=0;bit_state<=27;end
                27: //读 ACK
                    begin
                        if(phase0==1)sda_tem<=SDA;
                        if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
                        //下面 SDA 产生一个由低到高的跳变，结束一帧
                        if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=28;end
                    end
                28:
                    begin
                        if(phase1==1)sda_buf<=1; //SDA 从 0 到 1 的跳变，发送停止位 STOP
                        if(phase3==1)
                            begin bit_state<=0;eeprom_state<=0;DataLED<=DATA_IN;end
                            //DataLED 为送数码管显示的数
                    end
                default: begin bit_state<=0;eeprom_state<=0;end
            endcase
        end

2: //读 EEPROM 操作（其时序参见图 9-18 中的(b)图）
        begin
            if(phase0==1)scl_buf<=1;
            else if(phase2==1)scl_buf<=0;
            //1 到 0 跳变，产生开始 START 信号
            case(bit_state)
                0: //写 EEPROM 从机地址的第 0 位，因为 i2c_addr 从机地址是从高位开始的
                    begin
                        if(phase1==1)begin sda_buf<=0;Flag_RW<=1; end
                        if((phase3&Flag_RW)==1)
                            begin sda_buf<=i2c_addr[6];Flag_RW<=1;bit_state<=1;end
                    end
                1: //从机地址的第 1 位
                    if(phase3==1)begin sda_buf<=i2c_addr[5];Flag_RW<=1;bit_state<=2;end
                //略
                6: //从机地址的第 6 位
                    if(phase3==1)begin sda_buf<=i2c_addr[0];Flag_RW<=1;bit_state<=7;end
                7: //表示写从机（控制字节 CONTROL BYTE 中的 RW’为 0）
                    if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=8;end
                8: //准备读 ACK
                    if(phase3==1)begin Flag_RW<=0;bit_state<=9;end
```

```
9: //读 ACK
    begin
        if(phase0==1)sda_tem<=SDA;
        if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
        if(phase3==1)begin sda_buf<=eeprom_addr[7];Flag_RW<=1;bit_state<=10;end
    end
10: //写地址 bit1
    if(phase3==1)begin sda_buf<=eeprom_addr[6];Flag_RW<=1;bit_state<=11;end
//略
16: //写地址 bit7
    if(phase3==1)begin sda_buf<=eeprom_addr[0];Flag_RW<=1;bit_state<=17;end
17: //准备读 ACK
    if(phase3==1)begin Flag_RW<=0;bit_state<=18;end
18: //读 ACK
    begin
        if(phase0==1)begin sda_tem<=SDA;end
        if((phase1==1)&&(sda_tem==1))begin eeprom_state<=0;end
        if(phase3==1)begin sda_buf<=1;Flag_RW<=1;bit_state<=19;end
    end
//再次发送从机地址，即图 9-18(b)图中的第二个 CONTROL BYTE
19: //从机地址的第 0 位
    begin
        if(phase1==1)begin sda_buf<=0;Flag_RW<=1; end
        if((phase3&Flag_RW)==1)
            begin sda_buf<=i2c_addr[6]; Flag_RW<=1;bit_state<=20;end
    end
20: //从机地址的第 1 位
    if(phase3==1)begin sda_buf<=i2c_addr[5];Flag_RW<=1;bit_state<=21;end
//略
25: //从机地址的第 6 位
    if(phase3==1)begin sda_buf<=i2c_addr[0];Flag_RW<=1;bit_state<=26;end
26: //表示读从机（控制字节 CONTROL BYTE 中的 RW’为 1）
    if(phase3==1)begin sda_buf<=1;Flag_RW<=1;bit_state<=27;end
27: //准备读 ACK
    if(phase3==1)begin Flag_RW<=0;bit_state<=28;end
28: //读 ACK
    begin
        if(phase0==1)sda_tem<=SDA;
        if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
        if(phase3==1)begin Flag_RW<=0;bit_state<=29;end//准备读数据
    end
```

```
                    //开始读数据
                    29:
                        begin
                            if(phase1==1)RdData[7]<=SDA;
                            if(phase3==1)bit_state<=30;
                        end
                    30:
                        begin
                            if(phase1==1)RdData[6]<=SDA;
                            if(phase3==1)bit_state<=31;
                        end
                    //略
                    36:
                        begin
                            if(phase1==1)RdData[0]<=SDA;
                            if(phase3==1)bit_state<=37;
                        end
                    37: //进行 ACK 应答
                        if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=38;end
                    38:
                        begin
                            if(phase1==1)sda_buf <= 1; //SDA 从 0 到 1 的跳变，发送停止位 STOP
                            if(phase3==1)
                                begin bit_state<=0;eeprom_state<=0;DataLED<=RdData[3:0];end
                        end
                    default: begin bit_state<=0;eeprom_state<=0;end
                endcase
            end
        default: eeprom_state<=0;
    endcase
end
end
```